Practical Lab — Local AI with Ollama

Running and Developing Local AI Applications

Final project: **FSTT Local AI Assistant**
(Flask · Ollama · local RAG · conversation history · structured outputs)

Author: Fadoua El Kebraoui
Level: PhD student
Date: October 2026

Environment: Ubuntu Linux · Intel Raptor Lake laptop (integrated UHD Graphics, no dedicated GPU) · 16 GB RAM · Python 3.12 · JupyterLab · Ollama (CPU-only mode)
Models: qwen3.5:4b · gemma3:4b (Q4_K_M quantization)

1. Introduction and objectives

Cloud language models (ChatGPT, Claude, Gemini) are accessed through the Internet: the application sends its data to a remote API and pays per token. **Local AI** follows a different architecture: the model runs on the user's own computer, and applications communicate with it through a local service.

```
Python App → localhost:11434 → Ollama → Local LLM → CPU / GPU
```

The objectives of this lab were to install Ollama, run and compare local language models, use them from Python through a REST API, build a chatbot with memory, obtain structured (JSON) outputs, ground the answers in local documents (Mini-RAG), analyse an image with a multimodal model, measure performance, and finally build a complete local web assistant. This report presents the work carried out, the measured results and the answers to the report questions.

2. Environment and installation

2.1 Hardware

The hardware determines which models can run locally. It was checked with `free -h` and `lspci`:

Component	Value	Consequence
RAM	16 GB (about 9 GB free after closing applications)	4-billion-parameter models (about 3 GB) fit comfortably
GPU	Intel UHD Graphics (integrated), no NVIDIA/AMD GPU	Ollama runs in CPU-only mode: slower inference
OS / tools	Ubuntu, Python 3.12, virtual environment, JupyterLab	Notebook executed locally (not on Colab), so that it can reach Ollama

2.2 Installation of Ollama

Ollama was installed with the official script (`curl -fsSL https://ollama.com/install.sh | sh`). The installer created a systemd service, so the server starts automatically and exposes its API on `127.0.0.1:11434` (the page *http://127.0.0.1:11434* displays "Ollama is running"). As expected, it reported that no NVIDIA/AMD GPU was detected and that Ollama would run in CPU-only mode.

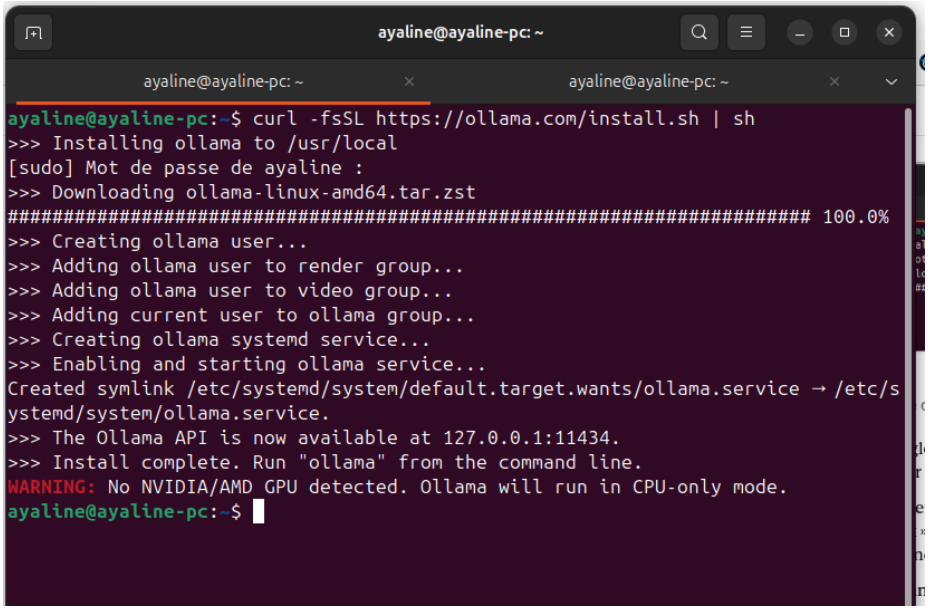


Figure 1 — Installation of Ollama (01_installation.png)

2.3 Choice of the models

The lab proposes *qwen3.5* and *gemma3*. Given the absence of a GPU and the 16 GB of RAM, the **4-billion-parameter versions** were selected: `qwen3.5:4b` (3.3 GB) and `gemma3:4b` (3.4 GB, also vision-capable, used in Part 14). A lighter model, `qwen3:1.7b` (1.4 GB), was also downloaded as a fallback. Since Ollama identifies models by their exact tag, the code of the lab was adapted to use `qwen3.5:4b` instead of the generic name `qwen3.5`.

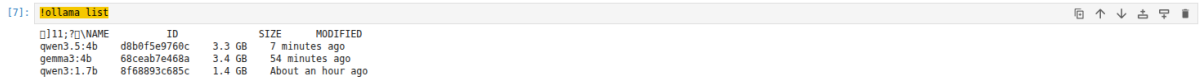


Figure 2 — Downloaded models (ollama list) (02_model.png)

3. First local LLM, prompts and model management

A helper function `generate()` sends requests to `/api/generate` and measures the wall-clock time. The same question about artificial intelligence, asked for a general audience, a 10-year-old child and a computer science professor, produced answers adapted to each audience: the model output depends on the whole prompt (role, audience, constraints). Inference is performed entirely on the local machine; no cloud API is involved.

The prompt-engineering exercise (Part 4) compared a detailed, constrained prompt on Directed Acyclic Graphs (definition, example, adjacency matrix, application, Python code) with a short prompt for a first-year student: the **role** sets the tone and expertise, the **context** and **audience** set the level, the **constraints** and **output format** structure the answer, and **specificity** reduces vague or generic answers.

For model management (Part 5), `ollama list` / `/api/tags` show the models **stored on disk** (name, size, family, 4.2B and 3.9B parameters, Q4_K_M quantization), whereas `ollama ps` / `/api/ps` show the models **loaded in memory**. The first call to `ollama ps` was empty: Ollama automatically unloads a model after 5 minutes of inactivity (`keep_alive`) to free memory, and reloads it at the next request.

4. Comparison of two local models (Part 6)

Both models received the same prompt: write a Python function that determines whether a number is prime, with type hints, a docstring, an explanation of the algorithm and its time complexity. The code generated by each model was **automatically extracted and tested** against a reference implementation on all integers from -5 to 2000.

Model	Response time (s)	Automatic test	Response quality /5	Code correctness /5
qwen3.5:4b	137.4	correct on -5..2000	5	5
gemma3:4b	103.9	correct on -5..2000	5	5

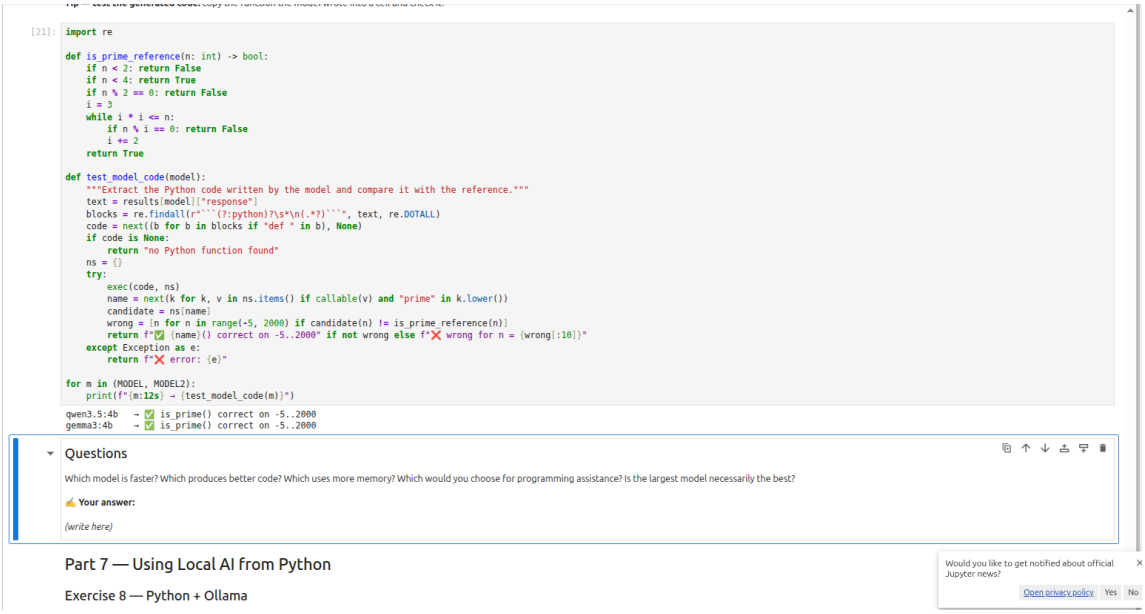


Figure 3 — Automatic verification of the generated functions against the reference

Both models produced correct code (trial division up to the square root of n, complexity O(√n)) and complete answers. gemma3:4b was about 25 % faster; qwen3.5 is a reasoning model whose thinking phase (enabled here) adds tokens before the answer. With similar sizes, the slightly larger model was not better, only slower.

5. Using local AI from Python and building a chatbot (Parts 7–9)

A minimal script (`local_ai_lab/app.py`) sends an HTTP POST request to `http://localhost:11434/api/generate` and prints the answer. The returned JSON also contains metadata: `total_duration`, `load_duration`, `eval_count` (generated tokens) and `eval_duration`. Since the computer has no dedicated GPU, the thinking mode of qwen3.5 was disabled (`"think": False`) in several cells to reduce response times.

```
Python → HTTP POST → localhost:11434 → Ollama → Local LLM → response → Python
```

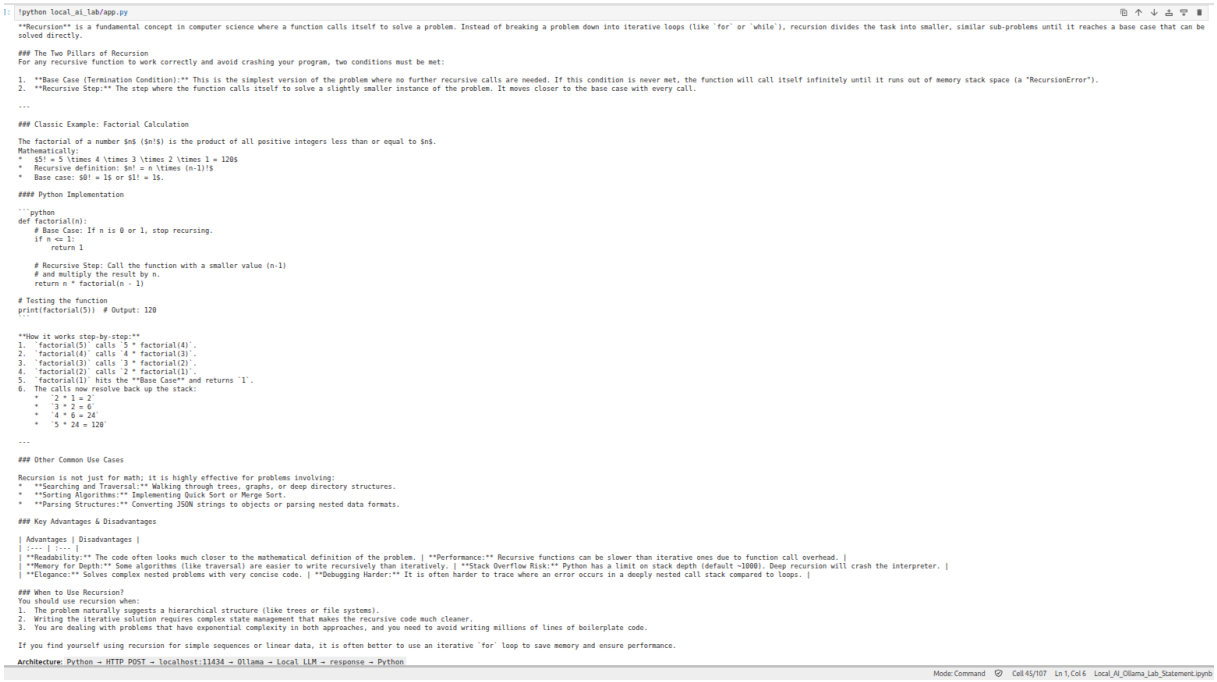


Figure 4 — Python client calling the local model (04_python.png)

A first chatbot treated each question independently. Memory was then added by sending the **whole conversation history** with each request. In the test, the model answered "Your name is Ahmed" after being told "My name is Ahmed." The printed prompt shows why: the previous exchange is included in the input. **The LLM itself is stateless**; the memory is managed by the Python `history` list, and clearing it makes the model "forget". With plain-text history the model answered "That's right!", a sign that it misinterpreted the format; the `/api/chat` endpoint, with structured `system`/`user`/`assistant` messages, simply answered "Your name is Ahmed."

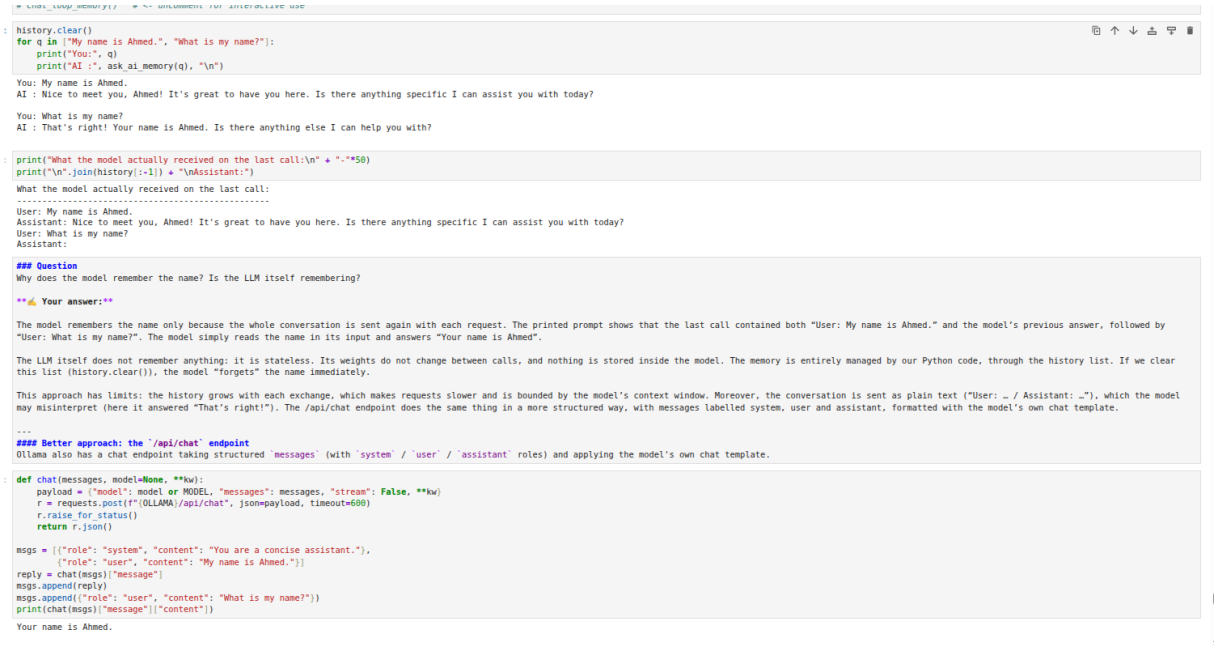


Figure 5 — Chatbot with conversation memory (03_chatbot.png)

6. Local AI as a code assistant (Part 10)

- **Calculator (318.7 s, thinking enabled):** the model identified the division by zero (`divide(10, 0)` throws an `ArithmeticException`), the truncation of integer division (7 / 2 = 3), and proposed a corrected version with an explicit check and a `divideDouble` method. Two

small errors appeared: the Javadoc mentions an exception for negative divisors, which the code does not implement, and `@see #add, #subtract` refers to methods that do not exist (a small hallucination).

- **Student (139.5 s):** the model found the expected bug: `age = age;` assigns the parameter to itself (variable shadowing); the fix is `this.age = age;`. It also added input validation and the missing `getAge()` method, and noted that age keeps its default value of 0, without explicitly pointing out that the constructor never initializes it.
- **Conclusion:** the local model is a useful reviewer, but slow with the thinking mode on a CPU, and its output must be checked by a developer.

7. Structured output (Part 11)

With the naive approach ("Return ONLY JSON"), the model returned valid JSON in 16.2 s (compared with 139 to 318 s for the code reviews with thinking enabled), but nothing guarantees it, which is why a defensive `extract_json` function is needed. With Ollama's **structured outputs** (a JSON Schema passed in `format`), the generation is constrained: all required fields are present, the types are respected (`Type of grade: <class 'float'>`) and `risk` can only be "low", "medium" or "high". Structured output turns the LLM into a reliable software component with a clear contract.

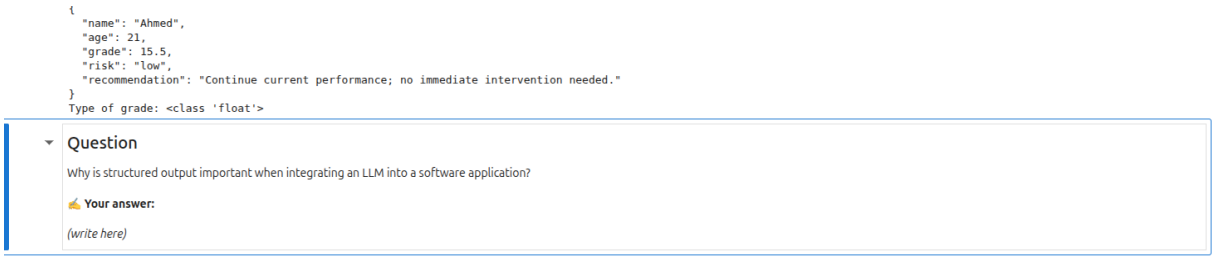


Figure 6 — Structured output constrained by a JSON Schema

8. Document assistant and Mini-RAG (Parts 12–13)

Asked to answer *only* from a short course document, the model answered "What is reinforcement learning?" with a sentence taken from the document (3.7 s), and "What is quantum computing?" with exactly "Information not available." (2.5 s), without using its training knowledge.

The Mini-RAG adds a local keyword retriever (TF-IDF, `rag.py`) over four documents (AI, machine learning, reinforcement learning, deep learning) plus the course file:

```
Documents → Local Retriever → Relevant Context → Ollama → Local LLM → Answer
```

Question	Retrieved document (score)	Answer	Time
What is reinforcement learning?	course.txt (1.08)	learns through interaction with an environment using rewards and penalties	4.5 s
Which hardware is needed to train neural networks with many layers?	deep_learning.txt (8.43)	GPUs.	2.7 s
What is supervised learning?	course.txt (1.08)	uses labelled data	3.0 s
Who won the 2018 World Cup?	none	Information not available.	1.8 s



Figure 7 — Mini-RAG results (05_rag.png)

The answers are grounded in the retrieved documents, and the out-of-scope question is correctly refused. A limit of keyword retrieval appears: for two questions, the short `course.txt` was selected instead of the more detailed dedicated documents, because the query words appear in several files with close scores. Retrieving two documents (as in the final application) or using semantic embeddings (`nomic-embed-text`) gives richer context.

9. Local vision (Part 14, optional)

The multimodal model gemma3:4b received an image encoded in base64 in the `images` field of the request. In 70.4 s, it correctly described a landscape of pink flowering trees (likely cherry blossoms) along a river, the flowers on the bank, the blue sky, the spring season and the peaceful atmosphere. It also reported a text "Unipatto" in the bottom right corner that is not visible in the image: a **visual hallucination**, showing that multimodal descriptions must be checked.

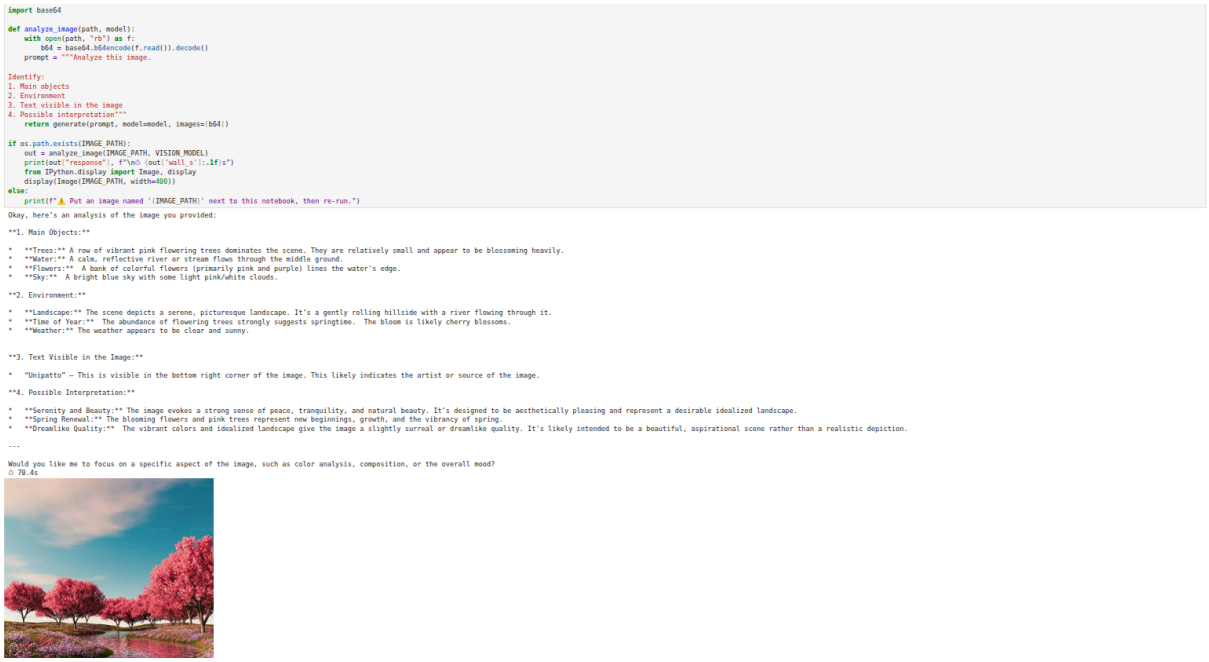


Figure 8 — Image analysis with gemma3:4b

10. Local AI web application (Part 15)

A first Flask application (`local-ai-web/app.py`, port 5000) receives a question from an HTML form, sends it to Ollama and displays the answer. It was launched from a terminal and tested in the browser; it answered, for example, a question asking to explain neural networks in three sentences.

11. Performance benchmark (Part 16)

Both models were benchmarked on the same prompt (a 500-word explanation of reinforcement learning), from a cold start, with thinking disabled. The values below are those of the final run:

Model	Disk (GB)	Loaded (GB)	VRAM (GB)	Load time (s)	Response time (s)	Tokens	Tokens/s	RAM used	Quality /5
qwen3.5:4b	3.32	3.15	0.0	10.21	154.49	751	5.25	89.1 %	5
gemma3:4b	3.35	2.89	0.0	22.67	95.68	799	11.10	81.8 %	5

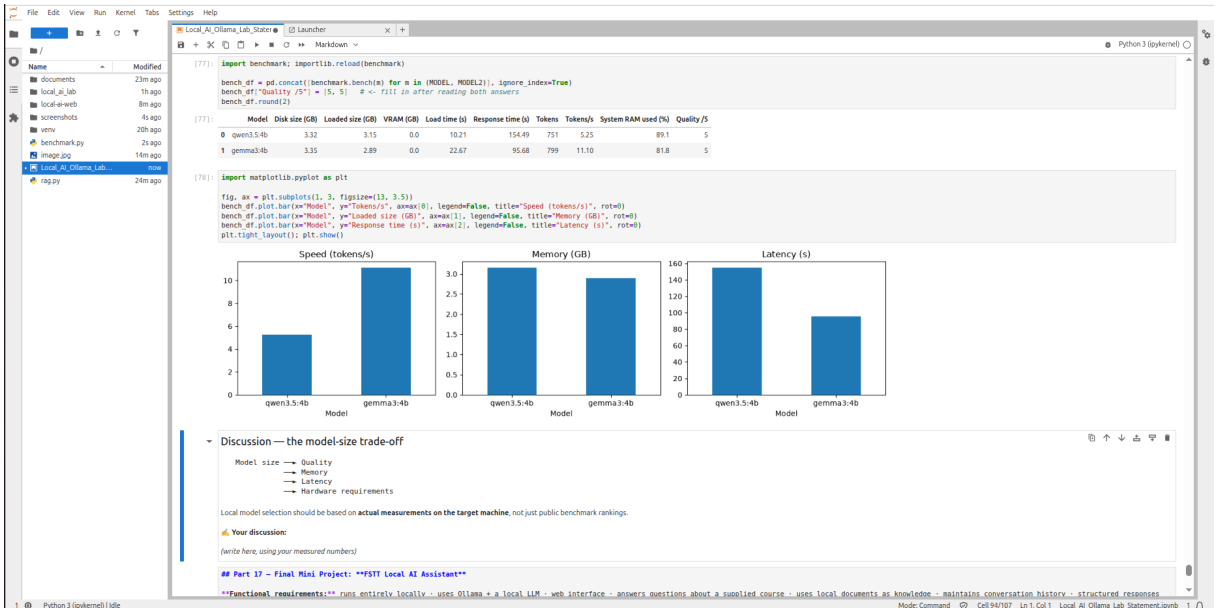


Figure 9 — Benchmark results and charts (speed, memory, latency)

- **Speed:** gemma3:4b generated 11.10 tokens/s against 5.25 for qwen3.5:4b, and answered in 95.7 s against 154.5 s, although its load time was longer (22.7 s vs 10.2 s, partly due to its vision components).
- **Variability:** in a first run of the same benchmark, qwen3.5:4b reached 10.95 tokens/s (75.7 s) and gemma3:4b 12.94 tokens/s (85.5 s). The speed of qwen3.5 was halved when the system RAM was 89 % full: on a CPU-only machine, results depend on free memory, other programs and processor temperature, so measurements should be repeated and averaged.
- **Memory:** similar sizes on disk (about 3.3 GB, Q4_K_M); once loaded, gemma3:4b used slightly less memory (2.89 GB vs 3.15 GB). VRAM = 0: everything runs on the CPU.
- **Quality:** both texts were coherent and relevant (5/5).

Trade-off: model size increases potential quality, but also memory, latency and hardware requirements. With two models of the same size class, the differences come from the architecture, the load time, the settings (the thinking mode) and the state of the machine. Local model selection must be based on measurements on the target machine.

12. Final mini project: FSTT Local AI Assistant (Part 17)

The final application combines all the previous components and runs entirely locally:

```
Browser → Flask/Python → [conversation history + documents → retriever] → Ollama → Local LLM
```

- web interface (HTML/JavaScript) served by Flask on `127.0.0.1:5001`;
- TF-IDF retriever over the `documents/` folder (two documents per question);
- conversation history (up to 6 exchanges) sent through `/api/chat`, with a Reset button;
- optional structured answers constrained by a JSON Schema (topic, summary, key points, difficulty);
- display of the response time, number of tokens, speed and sources for each answer;
- server bound to localhost only, for security.

Automatic test results:

Question	Feature tested	Result	Sources	Time
What is reinforcement learning?	RAG	Detailed answer (agent, state, action, reward, policy, Q-learning)	course.txt, reinforcement_learning.txt	4.74 s
And how is it different from supervised learning?	History	"it" correctly understood as reinforcement learning	course.txt, machine_learning.txt	5.92 s
Summarize deep learning	Structured JSON	Valid JSON with topic, summary, key points, difficulty	deep_learning.txt, course.txt	23.29 s
What is quantum computing?	Out of scope	"Information not available."	none	10.44 s

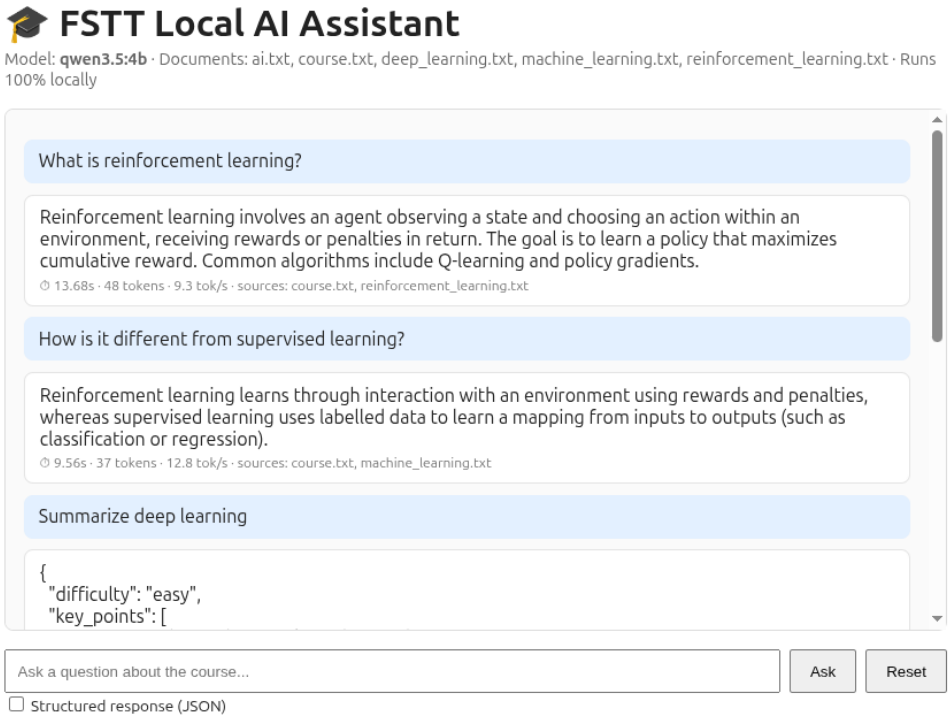


Figure 10 — FSTT Local AI Assistant (06_webapp.png)

Retrieving two documents corrected the limit observed in the Mini-RAG: the detailed `reinforcement_learning.txt` is now included and the answer is much richer. One limit of

structured outputs was observed: when the JSON option is checked for an out-of-scope question, the schema forces the model to fill every field, so it returns an empty summary and an invented difficulty instead of "Information not available." Adding a field such as `"available": true/false` to the schema would solve this.

13. Answers to the report questions (Part 18)

Q1. What is Local AI?

Local AI means running an artificial intelligence model, such as a large language model, **directly on one's own computer** instead of calling a remote cloud service. Inference is performed by the local CPU or GPU, and the data never leaves the machine. In this lab, Ollama ran qwen3.5:4b and gemma3:4b on a laptop, accessible through `localhost:11434`, without depending on an Internet connection.

Q2. Difference between a Cloud LLM and a Local LLM?

A **cloud LLM** (ChatGPT, Claude, Gemini) runs on the provider's servers: the application sends its data over the Internet to an API, usually pays per token, and benefits from very large and capable models. A **local LLM** runs on the user's machine: no Internet connection is needed, there is no per-token cost and the data stays private, but the model must fit in the available memory, so it is usually smaller, less capable and slower, especially without a GPU (about 5 to 13 tokens/s on our CPU-only laptop).

Q3. What role does Ollama play?

Ollama is the **runtime and model manager** for local LLMs. It downloads models (`ollama pull`), stores them (`ollama list`), loads them into memory when needed and unloads them after inactivity (`ollama ps`), performs inference on the CPU or GPU, and applies each model's chat template. Above all, it exposes a **local REST API** on `localhost:11434` (`/api/generate`, `/api/chat`, `/api/embed`, `/api/tags`, `/api/ps`), so that any program can use the same model.

Q4. What is model quantization?

Quantization reduces the **numerical precision of the model's weights**, for example from 16-bit floating-point numbers to about 4 bits per weight. The model becomes much smaller and faster, with a small loss of quality. Our two models use the **Q4_K_M** format: qwen3.5:4b has 4.2 billion parameters but takes only 3.32 GB on disk, whereas the same model in 16-bit precision would need about 8 GB. Quantization is what makes it possible to run such models on a laptop.

Q5. Why does model size affect memory consumption?

To generate text, all the model's weights must be loaded into memory (RAM or VRAM), since each generated token requires a computation over all of them. Memory therefore grows with the number of parameters multiplied by the size of each parameter (2 bytes in 16-bit precision, about 0.5 byte in 4-bit). In addition, the context (conversation history, documents) requires extra memory for the attention cache. In our benchmark, the 4-billion-parameter models used about 2.9 to 3.2 GB once loaded.

Q6. Why can a smaller model sometimes be preferable?

A smaller model loads faster, generates more tokens per second, uses less memory and can run on modest hardware, such as a laptop without a GPU. For simple or well-defined tasks (answering from documents, extracting JSON, writing short functions), its quality is often sufficient: in our tests, both 4B models wrote a correct prime-number function and answered the RAG questions correctly. This is why the 4-billion-parameter version of qwen3.5 was chosen rather than a larger one, given our 16 GB of RAM and the absence of a dedicated GPU.

Q7. Advantages of local inference?

Privacy: data never leaves the machine. **Offline operation**: the model works without an Internet connection. **No per-token cost** and no usage limits imposed by a provider. **Control**: choice of model, version and settings (thinking mode, structured outputs, keep_alive), with no dependence on an external service. **Easy integration** into any local application through the REST API.

Q8. Disadvantages of local inference?

Hardware requirements: enough RAM and ideally a GPU; on our CPU-only laptop, answers took from 2 seconds to more than 5 minutes (with thinking enabled). **Lower capability** than large cloud models. **Variable performance**: in our benchmark, qwen3.5 dropped from 10.95 to 5.25 tokens/s when the system RAM was 89 % full. **Maintenance and security**: installing, updating and securing the models and the server are the user's responsibility.

Q9. Explain the architecture of your Python application.

The FSTT Local AI Assistant follows the flow **Browser → Flask (Python) → retriever + conversation history → Ollama → local LLM**. The web page sends the question to the `/ask` route. The server uses the TF-IDF retriever (`rag.py`) to find the two most relevant documents in `documents/`, builds a list of messages with a system prompt, the recent history (up to 6 exchanges) and the question with its context, then calls `/api/chat`, with a JSON schema when a structured answer is requested. It returns the answer, the sources, the response time, the number of tokens and the speed, which the page displays. The history is stored in memory and cleared with the Reset button. In our tests, the application answered from the documents, understood a follow-up question thanks to the history, returned valid JSON, and refused an out-of-scope question.

Q10. Explain the role of the REST API.

The REST API is the **interface between applications and the model**. Ollama listens on `localhost:11434`, receives HTTP requests with a JSON body (model, prompt or messages, options such as `stream`, `format` or `think`) and returns JSON answers containing the generated text and metadata (`total_duration`, `load_duration`, `eval_count`, `eval_duration`). Thanks to this standard interface, the same model can be used from the terminal, a Python script, a Flask application or any other language. It follows the same principle as cloud APIs, which makes it easy to switch between a local and a cloud model.

Q11. What is RAG?

RAG (**Retrieval-Augmented Generation**) combines a **retrieval** step and a **generation** step: the system first searches a knowledge base for the documents relevant to the question, inserts them into the prompt as context, and asks the model to answer only from this context. The model can thus answer about information it was never trained on, and is less likely to hallucinate. In our Mini-RAG, the retriever found `deep_learning.txt` for a hardware question (answer: "GPUs") and no document for "Who won the 2018 World Cup?", so the model answered "Information not available." We also observed a limit of keyword retrieval (a less detailed document selected), improved in the final application by retrieving two documents; semantic embeddings would improve it further.

Q12. Difference between an LLM and a multimodal LLM?

A classic LLM only processes text. A multimodal LLM can also take other inputs, such as images: a vision encoder turns the image into visual tokens given to the language model together with the text prompt. In Part 14, gemma3:4b analysed a landscape in 70 seconds, correctly identifying the trees in bloom, the river and the spring atmosphere, but it also reported a text that was not visible in the image, showing that multimodal answers must be checked.

Q13. Compare two local models experimentally.

See the benchmark table in Section 11. Both models have a similar size (about 4 billion parameters, Q4_K_M) and run on the CPU only. In the final benchmark, gemma3:4b was faster (11.10 vs 5.25 tokens/s, 95.7 s vs 154.5 s) and used slightly less memory once loaded (2.89 vs 3.15 GB), but needed a longer load time (22.7 vs 10.2 s). In a first run, qwen3.5 reached 10.95 tokens/s, which shows that CPU measurements depend on the machine's state. In Part 6, both models wrote a correct prime-number function (5/5, verified on -5..2000), and gemma3 answered faster (104 s vs 137 s, with qwen3.5's thinking enabled). The quality of the answers was equivalent in all our tests.

Q14. Which model would you deploy on a computer with limited RAM? Explain.

Among the two models tested, I would deploy **gemma3:4b**: it uses the least memory once loaded (2.89 GB), it generated the most tokens per second (11.10), and its quality was equivalent. Its only drawback, a longer load time, can be avoided by keeping the model in memory (`keep_alive`). With even less RAM (8 GB or less), a smaller model such as `gemma3:1b` or `qwen3:1.7b` (about 1 to 1.5 GB) would be preferable, accepting a lower quality. In any case, the choice must be based on measurements on the target machine, with other applications closed to keep enough free memory.

Q15. What security risks exist when exposing a local LLM server to a network?

By default, Ollama listens only on `127.0.0.1`. If it is exposed to the network (for example with `OLLAMA_HOST=0.0.0.0`), several risks appear, because **Ollama has no authentication by default**: unauthorized use of the computing resources by anyone on the network; model management by third parties (the API also allows downloading or deleting models); exposure of prompts, documents and answers sent unencrypted over HTTP; prompt injection through malicious documents added to the knowledge base; and denial of service by long or numerous requests.

To reduce these risks: keep the server bound to localhost (as in our Flask application, which uses `host="127.0.0.1"`), block port 11434 with a firewall, and if remote access is needed, place a **reverse proxy** (such as Nginx) in front of it with **authentication and TLS encryption**, limit the request rate, and check the documents added to the knowledge base.

14. Conclusion

This lab showed that a complete AI application can run **entirely locally**, on a laptop without a dedicated GPU. Ollama made it possible to download, manage and serve two 4-billion-parameter models through a simple REST API, used from the terminal, Python scripts and web applications. The models wrote correct code, reviewed Java code, produced reliable JSON with structured outputs, answered from local documents with a Mini-RAG, analysed an image, and powered a final assistant combining retrieval, conversation history and structured answers.

The experiments also highlighted the limits of local inference: response times from a few seconds to several minutes on a CPU, a strong impact of settings such as the thinking mode, variable measurements depending on the machine's state, and hallucinations (non-existent methods, an invented text in an image). These observations confirm the central trade-off of local AI: privacy, offline operation and zero per-token cost, in exchange for smaller models and hardware constraints. The choice of a local model must therefore rely on measurements on the target machine, and its outputs must always be checked.

Appendix — Submission folder

```
LocalAI_Lab/
├── report.pdf
├── README.md
├── source/      app.py · chatbot.py · rag.py · benchmark.py · templates/index.html
├── documents/   ai.txt · course.txt · deep_learning.txt · machine_learning.txt ·
reinforcement_learning.txt
└── screenshots/ 01_installation.png … 06_webapp.png
```